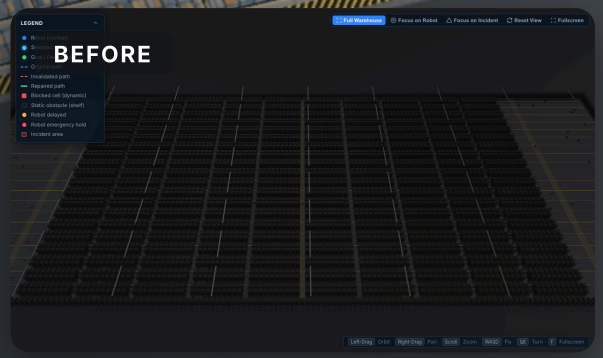
CHANGE REPORT · SESSION OF 30–31 OCTOBER 2026

The 3D warehouse viewer, rebuilt.

What changed in the AMR Digital Twin replay viewer, why it changed, and how to read the new visuals. Written so that someone new to the project can follow along.



Background: after. Inset: the same view before.

12 → 125
frames per second, 100 robots

2
phases shipped (world, incidents)

12
new focused source modules

01 · ORIENTATION

The project in one minute

A Python program simulates warehouse robots, throws problems at them, and repairs their routes. Everything it does is saved to a **replay file**. The 3D viewer only plays that file back: it never simulates anything itself. This session was entirely about the viewer.

Warehouse map
161 × 63 grid of 1 m cells: walls, 4,000 rack cells, open aisles.

→

Python pipeline (amr_repair/)

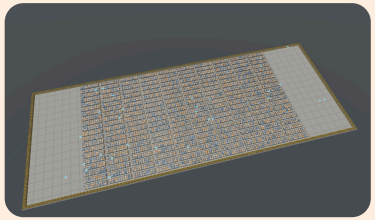
- 1 · Simulate robots
- 2 · Inject disruptions
- 3 · Detect incident
- 4 · Try strategy cache
- 5 · Ask LLM (fixed menu)
- 6 · Run solver
- 7 · Validate plan
- 8 · Fallback if needed

→

Replay file (JSON)
Every robot's cell and state at every step, plus every incident and its repair.

→

3D viewer ← this session



Words you will see

AMR: autonomous mobile robot, the small orange vehicles that carry goods.

Disruption: a blocked aisle cell, a robot that breaks down, or a robot that is delayed.

Invalidated path: the route a robot was following before the incident.

Validator: the deterministic checker. No plan runs without passing it.

Step: one tick of simulated time. A robot moves at most one cell per step.

Incident: one or more disruptions plus the robots whose routes they affect.

Repaired path: the new route chosen by the repair pipeline.

Baseline: the same scenario solved without the LLM or the cache, for comparison.

Where the viewer lives

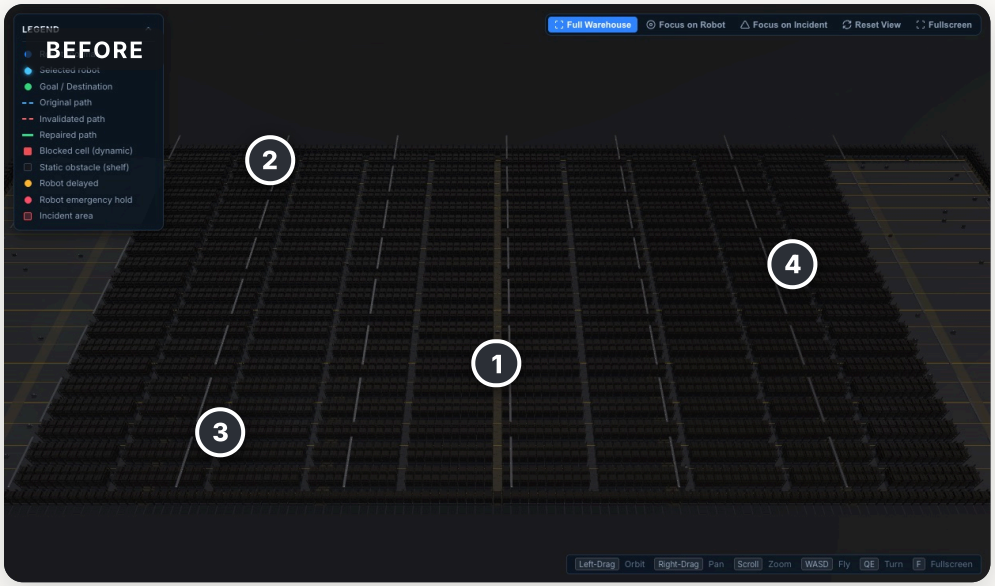
`viewer/` is a Vite + TypeScript app using Three.js (WebGPU). Start it with `cd viewer && npm run dev` and open `localhost:5173`.

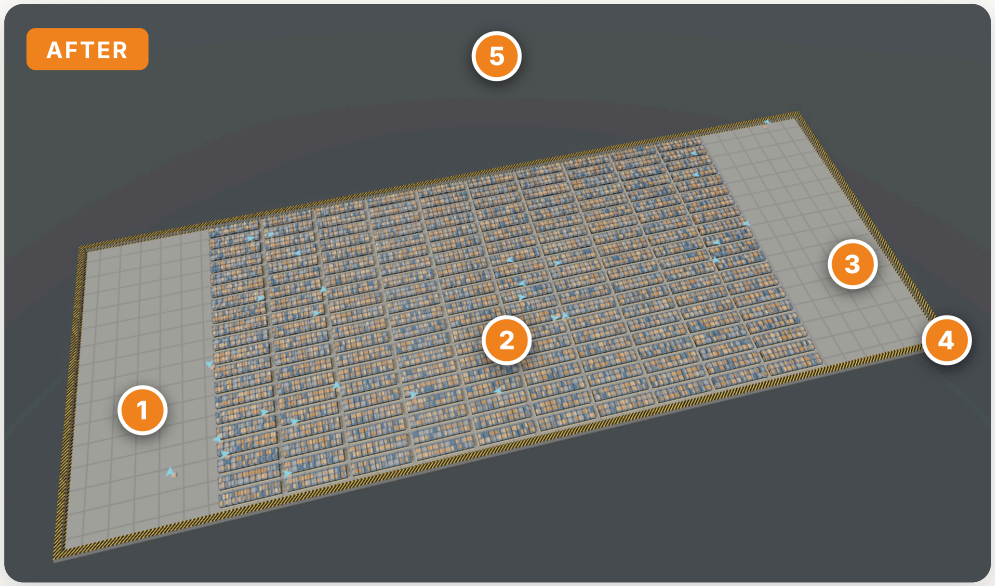
Replay files sit in `viewer/public/replays/`. Only `latest_experiment` comes from the real pipeline; the others were generated synthetically.

02 · THE HEADLINE

Before and after

The same full-warehouse view, same replay data, same laptop. The old scene was dark, hid the robots behind tall racks and ran at about 12 frames per second.





1. **Nearly black scene**
   No daylight, heavy fog, dark materials everywhere.
2. **Floating white bars**
   216 separate ceiling-light meshes and 16 point lights, all expensive to draw.
3. **Racks as black blobs**
   2.2 m racks hid the 1 m aisles the robots drive in.
4. **Robots invisible**
   Too small and dark to see at this distance.

1. **Foundation floor**
   Concrete plates and a faint 1 m grid show the planning grid.
2. **Low racks, open aisles**
   1.0 m racks in neutral steel-blue; the aisles stay visible.
3. **Robot markers**
   Arrows show every robot's state colour and heading from far away.
4. **Hazard-striped edge**
   The perimeter wall reads as the boundary of the floor.
5. **Daylight and haze**
   One sun, an HDR sky light and a soft sky gradient.

A stylised factory, inspired by factory-building games

The look borrows ideas (not assets or branding) from games like *Satisfactory*, where huge factories stay readable. Six rules guided every choice.

1 · Neutral world, saturated actors

The environment stays calm so the robots, the only things that move, pop.

- Concrete `#8E9398`
- Rack beam `#47729A`
- Upright `#2C3E52`
- Cardboard `#B8916A`
- Robot paint `#F0821E`

2 · Foundations and a visible grid

The floor is drawn as 4 m concrete plates with a faint 1 m grid, because the planner thinks in 1 m cells.



3 · Light carries state

Each robot's glowing ring and beacon show its state from the replay.

- Moving `#45C8FF`
- Idle `#3DDC84`
- Delayed `#FFB020`
- Replanning `#C86BFF`
- Failed `#FF3030`
- Selected `#FFFFFF`

4 · Holograms for plans

Anything that is a plan or a region, not a physical object, is drawn as a glowing hologram.

- Repaired route `#3CC8FF`
- Old plan `#FF4646`
- Medium area `#FFB020`
- Linked incident `#D65CFF`

5 · Safety markings mean edges

Yellow/black stripes only appear on hard limits: the outer wall, the floor band beside it and blocked cells. Yellow lane lines run along every rack.

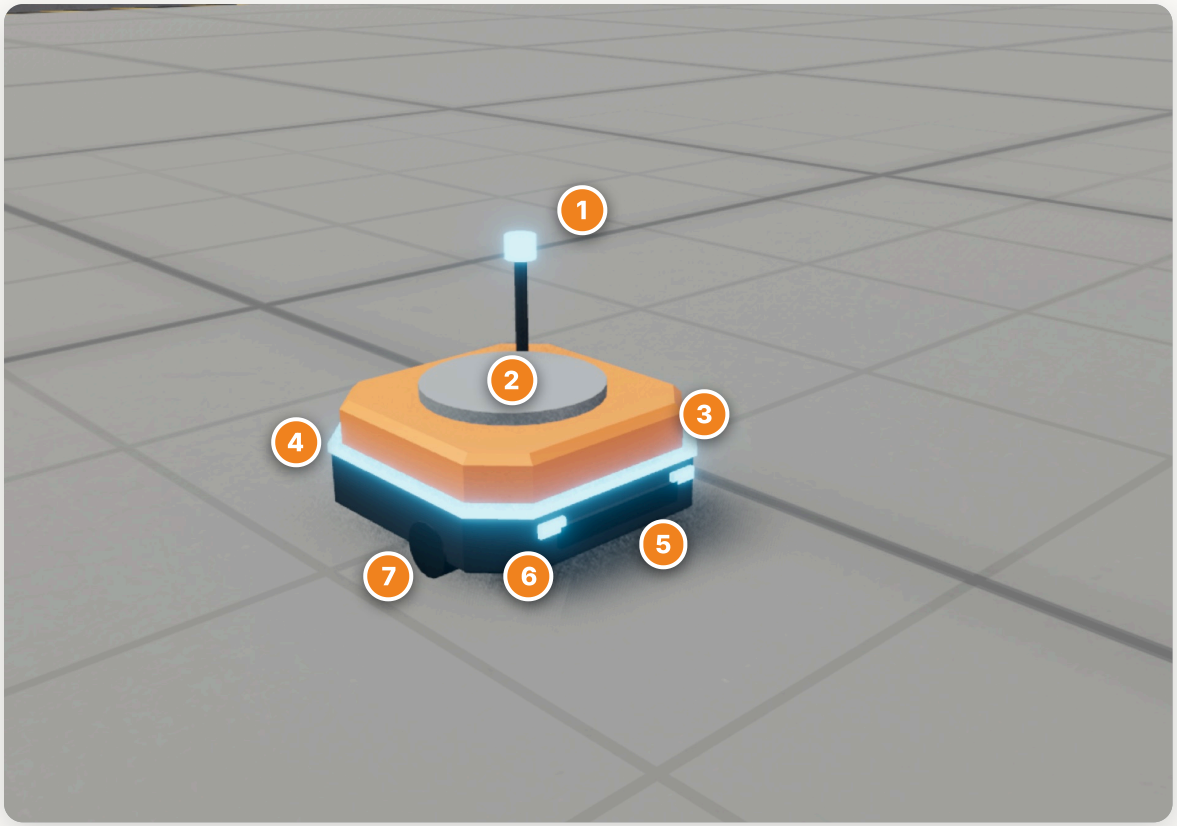
- Hazard `F2B705 / 1B1D21`
- Lane line `#F2C230`

6 · Honest by default

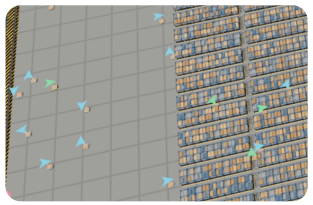
Every visual comes from the replay at the current step. When the replay lacks something (a robot's goal, when a blockage clears), the viewer says "not recorded" or marks it as an assumption, instead of inventing it.

Meet the new robot

A chunky silhouette that still reads when it is a few pixels tall. Every robot is drawn with three instanced meshes, so 100 or 200 robots cost the same number of draw calls as one.



1. **Status beacon**
   Glows in the robot's state colour, readable from far away.
2. **Lift plate**
   Where a tote would sit.
3. **Safety-orange shell**
   Bevelled edges catch the light. Dims when it is not part of a selected incident.
4. **Status light ring**
   Outlines the body from above in the state colour.
5. **Charcoal chassis**
   Chamfered base with a dark sensor window.
6. **Headlights**
   Mark the front, so heading is obvious.
7. **Drive wheels**
   Two drive wheels and two casters.

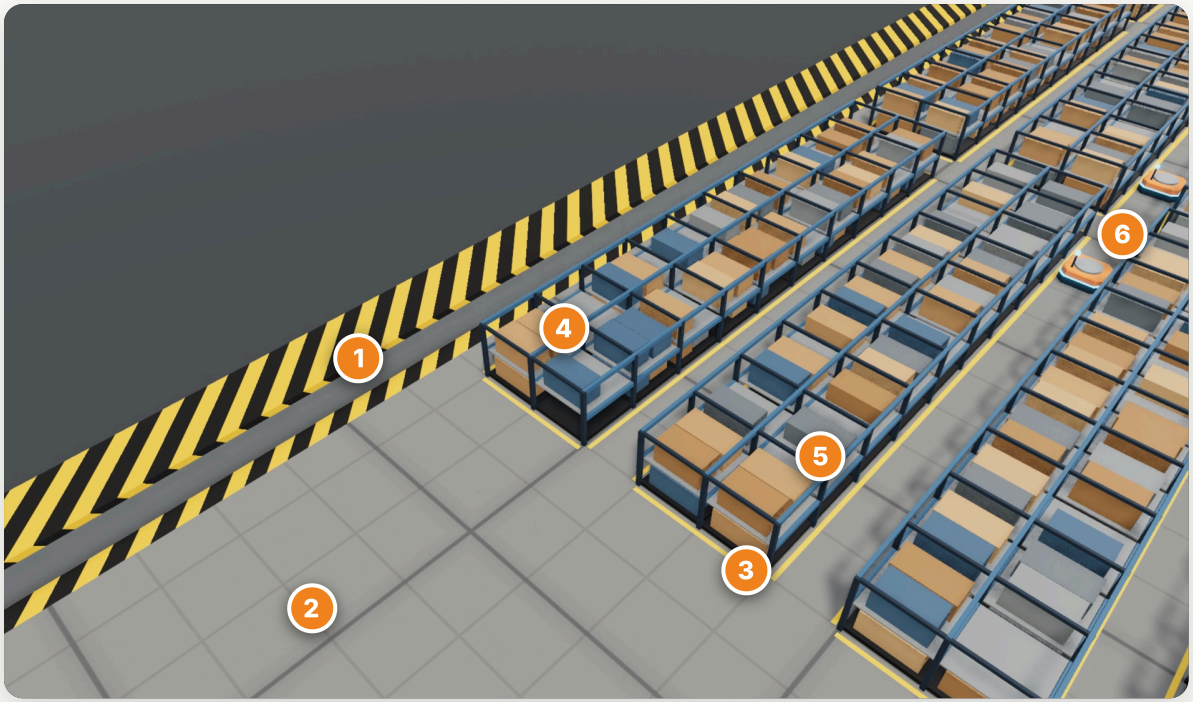


Zoomed out? Past about 30 m each robot gets a floating arrow in its state colour, pointing where it is heading. The arrows grow with distance and disappear in close-ups.

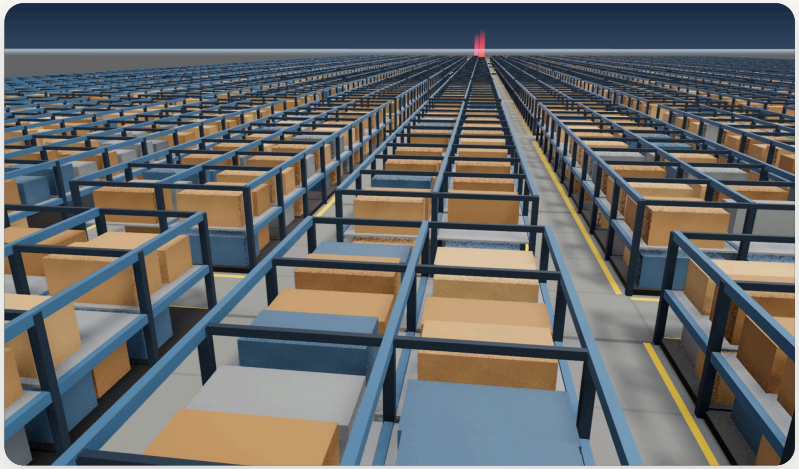
Size comes from the replay (30 or 100 robots), not a hard-coded 100. The old version stacked 70 phantom robots in a corner when a replay had 30.

The floor and the racks

The whole floor is one surface painted by a small GPU program that reads the warehouse map, so plates, grid, lane lines and safety bands cost a single draw call.



1. **Wall with hazard cap + floor safety band**
   The ring of wall cells around the map, drawn low so it never hides robots.
2. **Foundation plates and 1 m grid**
   The grid fades out with distance to avoid shimmering.
3. **Yellow lane lines**
   Painted wherever an aisle cell touches a rack.
4. **Rack bays**
   Steel-blue frames, 2 levels, 1.0 m tall. All 4,000 rack cells are one instanced mesh.
5. **Varied stock**
   Box colour and empty slots vary per rack cell inside the shader, at no extra cost.
6. **Robot in an aisle**
   Visible because the racks are low.



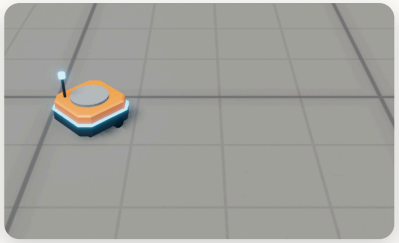
Down an aisle. The red pillar on the horizon is a disruption beacon (page 9).

How robots move now

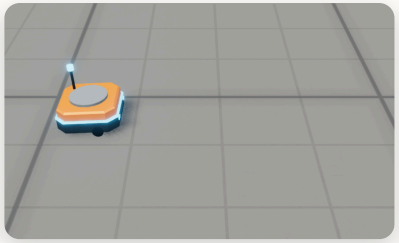
The replay only says which cell each robot is in at each step. Robot 3 below turns from north to east at step 16. The viewer now turns it in place first, then drives, and keeps its speed on straight runs instead of stopping at every cell.



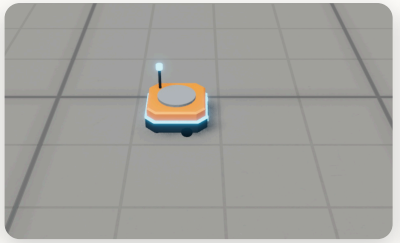
t = 15.55
Driving north



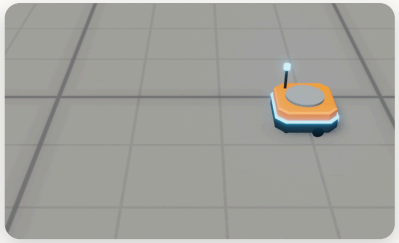
t = 16.15
Stopped, turning in place



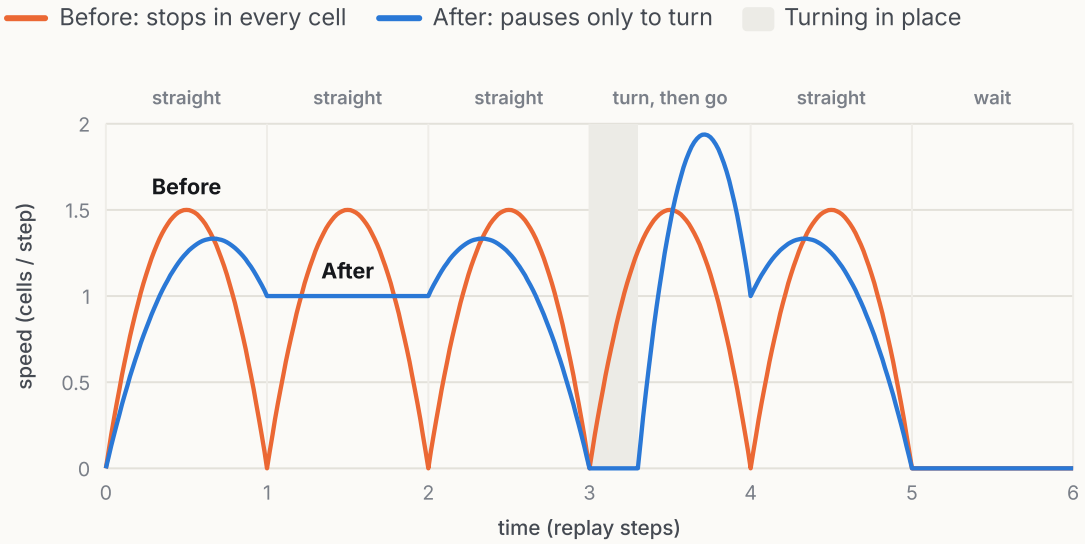
t = 16.32
Now facing east



t = 17.00
Pulling away, leaning back slightly



t = 18.45
Cruising, no stop at each cell



Checked against the real replays

- ✓ **No jumps.** Largest position change per 0.01 step is 0.03 cells (a teleport would be ≥ 1).
- ✓ **Exact at every step.** At each whole step every robot sits on the centre of its recorded cell.
- ✓ **Never drives while turning.** Zero samples moved and rotated at the same time.
- ✓ **Subtle weight.** Robots lean at most 2.9° when accelerating or braking.
- ✓ **Scrubbing is exact.** Every pose is computed from the frames at time t, so dragging the timeline backwards always shows the right picture.

Verified on all 30 robots × 61 steps of `latest_experiment` and 100 robots × 300 steps of `demo`, sampled every 0.01 step.

Reading an incident

Press `N` (next incident) and the camera flies to it, the warehouse dims, and the plan appears as holograms. Shown here: **inc_001** from the real pipeline run, where robot 23 is delayed and two robots get new routes.



Repaired route, arrows flow toward the goal

Old plan

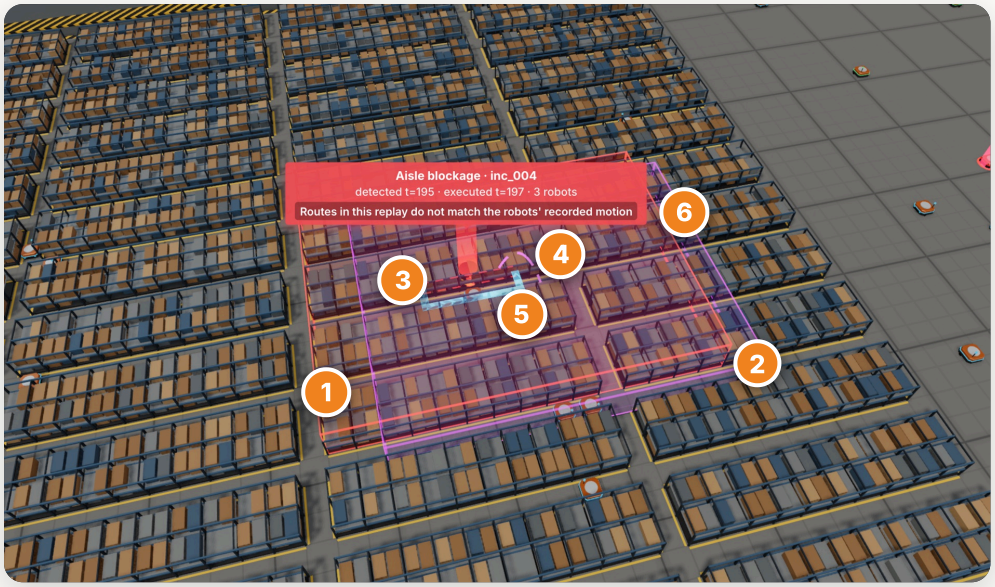
Congestion low / medium / high

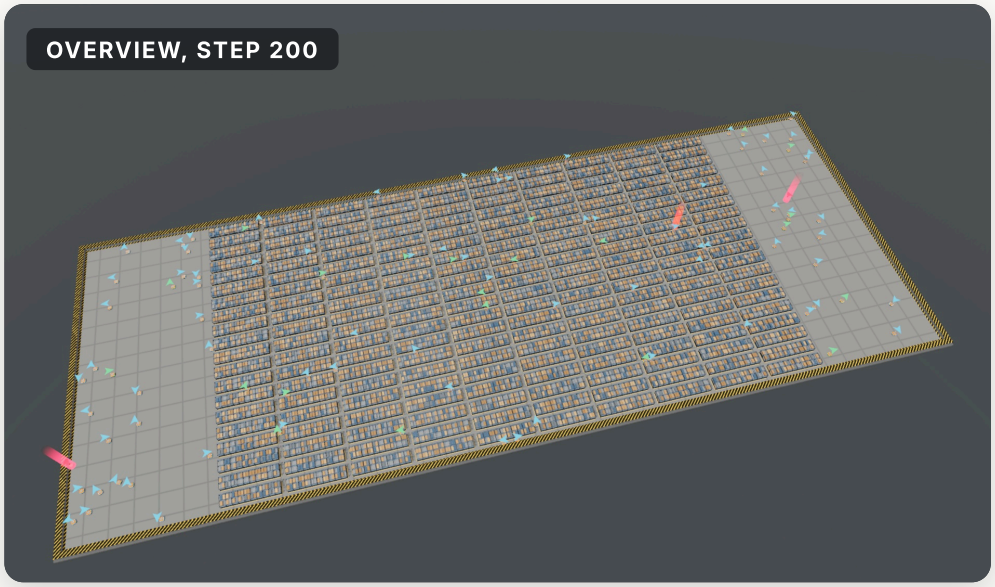
1. **Repaired route (cyan)**
   Where the robot will drive next. The part it has already driven dims as playback moves, using the replay step.
2. **Old plan (red dashed rails)**
   The route it was on before. Fades once the repair executes.
3. **Delay beacon (amber pillar)**
   Marks the delayed robot. Blocked cells get red pillars.
4. **Congestion region**
   A glowing box over the area the incident affects, coloured by level.
5. **Incident callout**
   Type, id, the steps it was detected and executed, robots affected.
6. **Robot callout**
   Click any robot: position, recorded goal, state and next cell.
7. **Focus mode**
   Racks and floor dim to about 45% so the plan stands out.

Routes also draw faintly through the racks (an x-ray pass), and get wider when zoomed out, so they are never hidden.

Linked incidents, and finding trouble from orbit

Some disruptions happen close together and affect each other. Every active disruption also gets a light pillar, so you can spot problems from the full overview.





1. **High-congestion region (red)**
   inc_004: a blocked aisle cell.
2. **Linked incident (magenta)**
   inc_005, a robot failure that interacts with it.
3. **Beacon on the blocked cell**
   Red light pillar plus a hazard-striped barrier block.
4. **Animated arc**
   Connects incidents the pipeline marked as interacting.
5. **Repaired route**
   Drawn exactly as stored in the replay.
6. **Honesty note**
   In this synthetic replay the routes do not match the robots' motion, so the viewer says so.

Beacons

- Blocked cell
- Failed robot (while the replay says failed)
- Delayed robot

Three pillars are visible above: one blocked cell and two robots that failed earlier and stay broken. Pillars grow with camera distance so they stay visible.

The replay does not record when a blocked cell clears, so the viewer shows it for its recovery window plus 15 steps. That is a viewer-side assumption and is labelled as one in the code.

09 · UNDER THE HOOD

How the code is organised now

The old viewer was one 2,007-line file. The 3D layer is now split into small modules with one job each; `main.ts` (1,079 lines) keeps app state, the camera and the existing UI wiring.

DATA

`types.ts`
Replay format, matching the Python models

`motion.ts`
Robot pose at any time t: turn, then drive

`palette.ts`
Every 3D colour in one place

WORLD

`world/grid.ts`
Reads the map: wall ring vs racks

`world/floor.ts`
Foundation floor shader

`world/racks.ts`
Racks + wall, instanced

`world/robots.ts`
Robot fleet and markers

`world/parts.ts`
Shared shape helpers

OVERLAYS & RENDERING

`world/disruptions.ts`
Barriers, rings, beacons

`world/incidents.ts`
Hologram plans, regions, arcs

`world/focus.ts`
Dims the world on selection

`render/stage.ts`
WebGPU renderer, sun, sky, post-processing

What happens every frame

1 · Advance the clock
Playback adds about 3.5 replay steps per second (times the speed setting).

2 · Pose every robot
`motion.ts` reads the frames around time t and returns position, heading and lean.

3 · Update the layers
Fleet, disruption beacons and the incident overlay redraw from the replay. Nothing builds up between frames.

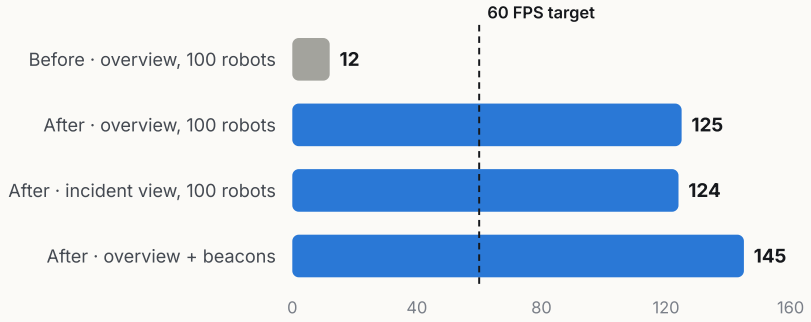
4 · Render
Half-size depth pass → ambient occlusion → main pass with 4× anti-aliasing → light bloom → screen.

Rendering follows the project rules: Three.js WebGPU renderer (falls back to WebGL2 by itself), one sun with soft shadows plus an HDR sky light, no ambient light, and post-processing limited to ambient occlusion and a light bloom.

Ten times faster, and closer to the project rules

Removing 16 point lights and 216 ceiling meshes, instancing everything that repeats and drawing the floor in one pass took the scene from about 12 to about 125 frames per second.

~12 FPS before **~125** FPS after



Uncapped frame rate in headless Chrome on an Apple M1 (integrated GPU), 1600×1000 viewport at 2× pixel ratio. A normal browser caps at the display refresh rate (60 Hz), so in practice it shows a steady 60.

PROJECT RULE (CLAUDE.MD)	BEFORE	NOW
Three.js WebGPU renderer	✕ WebGL	✓ WebGPU
One sun + HDR sky light, no ambient light	✕ 20 lights	✓
Post-processing: ambient occlusion + light bloom only	✕ none	✓
Robots and racks drawn as instanced meshes	◐ fixed at 100	✓ from replay
Turn before moving, ease in/out, never teleport	✕ snapped	✓ verified
Never invent data	✕ fake goals	◐ one labelled assumption
60 FPS with 200 robots on an integrated GPU	✕ 12 at 100	◐ 125 at 100; 200 not yet measured
UI text at least 14 px	✕	◐ callouts only

What we learned, and what comes next

Only the real replay is internally consistent

100%
route cells matching robot motion in `latest_experiment` (pipeline output)

0%
in `demo` and `scenario_*` (synthetic files)

In the synthetic files the routes sit far from the robots listed as affected, and the "baseline" robots move exactly like the LLM ones. Regenerating them from the pipeline would fix this.

Spotted in the Python pipeline (not changed)

- ! A route repair overwrites the robot's `delayed` state, so real replays never show a delayed robot.
- ! The strategy cache is saved between runs, so results depend on earlier runs, not just the random seed.
- ! `.env` only sets an OpenAI key, but the code sends it to Groq by default. The LLM calls likely fail and fall back to a fixed rule.

Phase 3, next: the interface

- ! Reconnect the panels that still show placeholder text: incident list, timeline markers, timestep readout, speed control, view toggles.
- ! Restyle the interface in charcoal and orange to match the 3D scene; bring all text to at least 14 px.
- ! Make the legend and the 3D colours agree (the legend still says repaired routes are green).
- ! Remove selector options that point at files that do not exist.

Try it yourself

`cd viewer && npm run dev`, then open `localhost:5173`.

`Space`	Play / pause
`N`	Next incident (camera flies there)
`1`	Whole warehouse
`3`	Follow the selected robot (click a robot first)
`Esc`	Clear the selection
`Scroll`	Zoom; robot arrows appear when you zoom out